



Reflection

CLIENT HANDOVER MANUAL

Part I Deployment & Installation Guide

Part II Administrator & Operations Manual

Contents

Deployment & Installation Guide.....	3
1 What Reflection does.....	4
2 Components and where they run.....	4
3 Requirements.....	4
4 Before installation: choose ports and firewall rules.....	5
5 Installation.....	5
6 Secure the configuration first.....	5
7 Configure a standard TCP relay.....	6
7.1 Mirror host: conf/mirror.json.....	6
7.2 Reflector host: conf/reflect.json.....	6
7.3 Start order and test.....	7
8 TCP, UDP, and multi-service launch commands.....	7
8.1 Multiple TCP mappings.....	7
Administrator & Operations Manual.....	9
9 Configuration reference.....	10
9.1 conf/reflect.json.....	10
9.2 conf/mirror.json.....	11
9.3 Worker and logging configuration.....	11
10 Optional extensions and data handling.....	11
11 Optional firewall control.....	12
11.1 Firewall administration commands.....	12
11.2 Geolocation firewall rules.....	13
12 Production service management.....	14
13 Operation and validation checklist.....	14
14 Troubleshooting.....	15

PART I

Deployment & Installation Guide

IN THIS PART

- 1 What Reflection does
- 2 Components and where they run
- 3 Requirements
- 4 Before installation: choose ports and firewall rules
- 5 Installation
- 6 Secure the configuration first
- 7 Configure a standard TCP relay
- 8 TCP, UDP, and multi-service launch commands

1 What Reflection does

Reflection publishes a TCP or UDP service that is reachable from a public mirror server, while the real service stays on a private reflector server. A reflector creates encrypted control/data channels to the mirror. When a client connects to the published mirror port, Reflection assigns one channel and connects to the configured origin service.

Client → public mirror:published port → encrypted Reflection channel → reflector → private origin service

Figure 1. Connection path from a client to the private origin service.

This is a transport-layer relay. It does not understand HTTP, TLS, databases, or application protocols; it forwards TCP streams or UDP datagrams. Use it only where a reverse TCP/UDP relay is the intended architecture.

2 Components and where they run

Table 1. Reflection components and where they run.

Component	Run on	Purpose
Mirror	Public/reachable host	Receives reflector control/data channels and opens the client-facing listener on demand.
Reflector	Network that can reach the origin	Connects to the mirror, maintains a TCP channel pool, and opens connections to the origin.
Origin service	Usually private	The existing TCP/UDP service being made reachable. It is not part of this repository.

3 Requirements

- Node.js with the built-in `net`, `dgram`, `crypto`, `cluster`, and `filesystem` modules. The repository has no `package.json` and no third-party Node dependency install step.
- Two hosts with network connectivity from reflector to mirror control/data ports.
- An origin service reachable from the reflector on the configured host and port.
- Permission to listen on the selected ports. Use ports above 1024 unless the service account has the required capability.
- The mirror and reflector must use the same `conf/crypt.json` values. Control traffic and TCP relay control messages use the shared key; the optional AES extension has its own `conf/aes.json` secret.

Check Node before deployment:

```
node --version
node -e "require('net'); require('dgram'); require('crypto'); console.log('Node
runtime OK')"
```

4 Before installation: choose ports and firewall rules

Choose non-conflicting ports. Do not expose the control, heartbeat, or firewall-control ports to the Internet. Restrict them to reflector/admin addresses. The published service port is the only port normally open to clients.

IMPORTANT · UDP

The implementation starts its heartbeat server with Node's TCP `net` API even for the UDP mode. Keep the heartbeat port reachable over TCP until a future release.

5 Installation

Perform these steps on both hosts.

1. Copy the complete reflection directory to a stable location, for example `/opt/reflection`.
2. Create a non-login service account, for example `reflection`.
3. Give that account ownership of the installation and ensure it can create `/opt/reflection/logs`.
4. Keep `conf/` readable only by the service account because it contains secrets.

6 Secure the configuration first

The supplied configuration contains sample/default secrets and example addresses. Treat it as a template, not production configuration.

1. Generate two new high-entropy secrets: one for `key`, one for `heartbeat_key`.
2. Put them in `conf/crypt.json` on both hosts, with identical values.
3. If using the AES traffic extension, replace `conf/aes.json` with the encrypted AES key expected by `extensions/aes.js`; otherwise do not enable that extension.

WARNING

Never put secrets directly in source control. Any string config value may instead use `env:VARIABLE_NAME`; Reflection resolves it from the process environment at startup.

Example `crypt.json` pattern using environment variables:

```
{
  "key": "env:REFLECTION_CONTROL_KEY",
  "heartbeat_key": "env:REFLECTION_HEARTBEAT_KEY",
  "fips_mode": false
}
```

Set those environment variables in the service manager, not in a world-readable shell file. Both endpoints must receive exactly the same values. If FIPS is enabled on the installation system, then change `fips_mode` flag as `true`.

7 Configure a standard TCP relay

7.1 Mirror host: `conf/mirror.json`

Set the bind addresses and shared control ports. A conservative example:

```
{
  "host": "0.0.0.0",
  "port": 2000,
  "heartbeat_tcp": 2001,
  "keep_alive": 1000,
  "timeout": 300000,
  "firewall_enabled_tcp": false,
  "firewall_enabled_udp": false
}
```

`port` is the Reflection control/data port, not the client-facing service port. The reflector asks the mirror to create the client-facing listener dynamically. `host` should be a specific public or private address where possible; use `::` only when intentionally accepting IPv6.

7.2 Reflector host: `conf/reflect.json`

For an origin service at `10.0.0.25:8080`, published as mirror port `1201`:

```
{
  "reflected_host": "10.0.0.25",
  "reflected_port": 8080,
  "mirror_host": "<mirror_ip>",
  "mirror_port": 2000,
  "mirror_heartbeat_tcp": 2001,
  "mirror_service_host": "0.0.0.0",
  "mirror_service_port": 1201,
  "mirror_force_listener": true,
  "pool_size": 5,
  "mirror_connect_timeout": 20000,
  "heartbeat_check_interval": 60000,
  "dhcp_check": true,
}
```

```
"logfile": "reflector_tcp.log.ndjson"
}
```

`reflected_*` identifies the origin from the reflector's network. `mirror_*` identifies the mirror control endpoint. `mirror_service_*` defines the public listener created on the mirror. `pool_size` is the number of idle TCP channels; it also caps the number of simultaneous TCP client sessions for this mapping. Size it for expected concurrency plus a small buffer.

For IPv4/IPv6 UDP, also set `mirror_hostudp4`, `mirror_hostudp6`, `mirror_service_host_udp4`, `mirror_service_host_udp6`, and `ipv6enabled` appropriately. Disable IPv6 (false) unless both hosts and firewall rules support it.

7.3 Start order and test

Start the mirror first, then the reflector:

```
cd /opt/reflection
node --env-file=.env mirror_tcp.js

cd /opt/reflection
node --env-file=.env reflect_tcp.js
```

NOTE

Do not regard an open TCP port alone as proof of success: confirm an actual application response.

```
curl -v http://<mirror_ip>:1201/
```

8 TCP, UDP, and multi-service launch commands

Table 2. Launch commands for each deployment mode.

Use case	Mirror command	Reflector command
TCP	<code>node mirror_tcp.js</code>	<code>node reflect_tcp.js</code>
UDP	<code>node mirror_udp.js</code>	<code>node reflect_udp.js</code>
Many TCP mappings	<code>node mirror_tcp.js</code>	<code>node massreflect_tcp.js</code>

8.1 Multiple TCP mappings

Use `conf/massreflect.json` when one reflector publishes multiple origins or multiple reflectors use the same mirror.

Each `reflected_hosts` string follows this syntax:

```
"origin-host:origin-port->published-bind-address:published-port"
```

NOTE

Can leave the `Mirror_hostsudpv6` blank if using ipv4.

```
[
  {
    "reflected_hosts": ["216.116.72.174:80->[::]:1201", "216.116.72.174:443->[::]:1202"],
    "mirror_host": "<mirror_ip>:<mirror_port>",
    "mirror_hostsudpv4": "<mirror_ipv4>:<mirror_port>",
    "Mirror_hostsudpv6": "<mirror_ipv6>:<mirror_port>",
    "mirror_heartbeat_tcp": 2001,
    "mirror_heartbeat_udp": 1002
  }
]
```


PART II

Administrator & Operations Manual

IN THIS PART

- 9** Configuration reference
- 10** Optional extensions and data handling
- 11** Optional firewall control
- 12** Production service management
- 13** Operation and validation checklist
- 14** Troubleshooting

9 Configuration reference

9.1 conf/reflect.json

Table 3. Reflector configuration keys.

Key	Meaning
reflected_host, reflected_port	Origin address and port, reachable by the reflector.
mirror_host, mirror_port	Mirror TCP control/data endpoint.
mirror_hostudp4, mirror_hostudp6	UDP mirror addresses.
mirror_heartbeat_tcp, mirror_heartbeat_udp	Mirror heartbeat ports.
mirror_service_host, mirror_service_port	TCP public bind address and port created on the mirror.
mirror_service_host_udp4, mirror_service_host_udp6	UDP public bind addresses.
mirror_force_listener	Replaces an existing listener for the mapping when true; avoid sharing a published port between reflectors.
pool_size	Idle TCP channel count / per-mapping concurrency ceiling.
mirror_connect_timeout	Control/UDP connection timeout in milliseconds.
heartbeat_check_interval	Heartbeat interval in milliseconds.
mirror_heartbeats_lost_for_exit	Consecutive failed beats tolerated before exit; supervisor should restart it.
dhcp_check, dhcp_check_interval	Exits when non-internal network interface addresses change; enable when a supervisor will restart after address changes.
reflector_channel_check_wait	Exit if the TCP channel pool is empty longer than this many milliseconds.
keep_alive	TCP keepalive delay in milliseconds; use -1 to skip.
ipv6enabled	Enables UDP IPv6 socket use.
udp_cleanup_interval	Idle UDP mapping cleanup time in milliseconds; -1 means no cleanup.

Key	Meaning
<code>logfile</code>	TCP reflector log filename below <code>logs/</code> .

9.2 `conf/mirror.json`

Table 4. Mirror configuration keys.

Key	Meaning
<code>host,</code> <code>port</code>	TCP mirror bind address and Reflection control/data port.
<code>hostudp4,</code> <code>hostudp6</code>	UDP bind addresses.
<code>heartbeat_tcp,</code> <code>heartbeat_udp</code>	Heartbeat server ports.
<code>keep_alive,</code> <code>timeout</code>	TCP channel keepalive and idle timeout.
<code>udp_cleanup_interval</code>	Idle UDP client mapping cleanup period.
<code>ipv6enabled</code>	Whether mirror UDP also binds IPv6.
<code>firewall_*</code>	Optional admission-control settings; see next section.

9.3 Worker and logging configuration

`cluster_reflector.json` and `cluster_mirror.json` set workers; 0 means use CPU count, subject to `min_workers`. More mirror workers add their worker index times `offset_window` to mirror, heartbeat, and firewall ports. In a multi-worker deployment, open and document each offset port; keep workers: 1 for a simple client installation.

`log.json` controls debug logging, synchronous writing, and the nominal `max_log_mb`. Logs are newline-delimited JSON in `logs/`. Rotate and retain them with the operating system's log facility; do not rely solely on the application's maximum-size setting.

10 Optional extensions and data handling

`conf/extensions.json` defines an ordered extension chain. The supplied default enables both traffic recorder and traffic logger.

Table 5. Extensions supplied with Reflection.

Extension	Configuration	Effect	Handover advice
trafficlogger	<code>trafficlogger.json</code>	Writes metadata and payload chunks to <code>logs/traffic.log</code> ; payloads are Base64 by default.	Disable in production unless it is required and retention/access controls are approved.
trafficrecorder	<code>trafficrecorder.json</code>	Writes raw inbound/outbound payload streams to <code>logs/capture.log.in</code> and <code>.out</code> .	Treat as sensitive capture data; normally disable.
aes	<code>aes.json</code>	Adds framed AES payload encryption for TCP only.	Add only after both ends use matching extension order/key and an end-to-end test succeeds.

Set both logger/recorder enabled values to false to stop payload capture. Ensure the `logs/` directory exists before enabling extensions.

NOTE

Extension order matters: outbound processing runs in reverse order, so both ends must have the same list/order.

To enable aes extension add `{"extension": "aes"}` in `extension.json`

11 Optional firewall control

When `firewall_enabled_tcp` or `firewall_enabled_udp` is true, a newly created mirror listener denies traffic unless the source IP is explicitly allowed or passes the configured geography rules.

WARNING

The firewall control port is powerful and must be network-restricted.

11.1 Firewall administration commands

These commands manage IP access to client-facing listeners on the mirror. Set `firewall_enabled_tcp: true` on the mirror and restrict the firewall-control port to trusted administrators.

```
node lib/firewall.js open CLIENT_IP PUBLISHED_PORT MIRROR_HOST FIREWALL_CONTROL_PORT
[swf_mode|heartbeat_mode]
```

Allows `CLIENT_IP` to connect to `PUBLISHED_PORT`. Use `all` instead of `PUBLISHED_PORT` to target every active listener.

- `sfw_mode`: automatically revokes access when that client's TCP session ends.
- `heartbeat_mode`: keeps access only while periodic heartbeat commands continue.

```
node lib/firewall.js close CLIENT_IP PUBLISHED_PORT MIRROR_HOST FIREWALL_CONTROL_PORT
```

Revokes `CLIENT_IP`'s access to `PUBLISHED_PORT` and disconnects that IP's currently active sessions. Use `all` to revoke it from every active listener.

```
node lib/firewall.js heartbeat CLIENT_IP PUBLISHED_PORT MIRROR_HOST  
FIREWALL_CONTROL_PORT
```

Refreshes an existing rule created with `heartbeat_mode`. It does not create or reopen a rule. If these heartbeats stop, the mirror removes the allowance after `firewall_timeout_for_heartbeat_mode_autoclose`.

```
node lib/firewall.js check CLIENT_IP PUBLISHED_PORT MIRROR_HOST FIREWALL_CONTROL_PORT
```

Checks whether `CLIENT_IP` would currently be allowed to access `PUBLISHED_PORT`, considering explicit firewall rules and configured geofencing.

Example:

```
node lib/firewall.js open 203.0.113.10 9898 mirror.example.com 2003 heartbeat_mode  
node lib/firewall.js heartbeat 203.0.113.10 9898 mirror.example.com 2003  
node lib/firewall.js check 203.0.113.10 9898 mirror.example.com 2003  
node lib/firewall.js close 203.0.113.10 9898 mirror.example.com 2003
```

NOTE

Geofencing may contact the configured external HTTP geolocation endpoint. Consider privacy, egress controls, and dependency availability before enabling it.

11.2 Geolocation firewall rules

Geolocation is configured on the mirror in `conf/mirror.json`; there is no separate firewall CLI command for it.

```
{  
  "firewall_geofencing_additive": ["ML", "SG", "JP", "private_blocks", "localhost"],  
  "firewall_geofencing_subtractive": ["RU", "CN"],  
  "firewall_geo_server": "www.geoplugin.net",  
  "firewall_ip_cache_size": 2000000  
}
```

- `firewall_geofencing_additive` — automatically allows clients whose location matches an entry, even if no explicit open command was issued.

Valid country entries are ISO two-letter country codes, such as "IN", "SG", or "US".

- `firewall_geofencing_subtractive` — always denies matching client locations. This takes precedence over an explicit open firewall rule.
- `firewall_geo_server` — hostname used to look up public client IP locations. The current implementation calls it over HTTP on port 80.
- `firewall_ip_cache_size` — maximum number of IP-to-location lookups retained in memory.

Example: allow India, Singapore, and private-network addresses by default, but block Russia and China:

```
{
  "firewall_enabled_tcp": true,
  "firewall_geofencing_additive": ["IN", "SG", "private_blocks", "localhost"],
  "firewall_geofencing_subtractive": ["RU", "CN"],
  "firewall_geo_server": "www.geoplugin.net"
}
```

NOTE

Restart the mirror after changing these settings. If the geolocation service cannot be reached, an unknown public IP is not automatically allowed by the additive list; use open for explicit access where needed.

12 Production service management

Run each long-lived command under `systemd`, a container supervisor, or an equivalent process manager. Reflection intentionally exits when it loses connectivity, misses required heartbeats, or detects an address change; a supervisor must restart it.

13 Operation and validation checklist

- ☐ Check that the mirror process is running and listening on the configured control port.
- ☐ Check that each reflector process is running and that its log shows an active mirror listener and pool/channel activity.
- ☐ Test one application request through every published service port from the intended client network.
- ☐ Review `logs/*.ndjson` for [access], connection errors, heartbeat failures, and firewall blocks.
- ☐ Check disk usage under `logs/`, particularly if traffic capture is enabled.
- ☐ After any secret or configuration change, restart the mirror and reflector together and repeat the end-to-end test.

Useful checks:

```
ss -lntup
tail -f /opt/reflection/logs/mirror_tcp.log.ndjson
tail -f /opt/reflection/logs/reflector_tcp.log.ndjson
```

14 Troubleshooting

Table 6. Symptoms, likely causes, and corrective action.

Symptom	Likely cause and action
Reflector cannot start listener	Verify mirror hostname/port, network ACLs, matching <code>crypt.json</code> , and that the requested published port is free.
Mirror says no reflector is available	The reflector is down, cannot reach the mirror, has a key mismatch, or all TCP pool channels are allocated. Check reflector logs and raise <code>pool_size</code> if concurrency is expected.
Listener disappears	The mirror closes a listener when its last reflector channel closes. Restore the reflector; supervisor restart should recreate it.
Heartbeat exits a reflector	Verify the mirror heartbeat port and matching heartbeat key. Check <code>heartbeat_check_interval</code> , route/firewall, and the failure threshold.
Traffic reaches mirror but not origin	From the reflector host, test <code>reflected_host:reflected_port</code> directly. Verify DNS, routing, and origin ACLs.
UDP has no response	Check UDP ACLs in both directions, IPv4/IPv6 fields, cleanup timeout, and the TCP heartbeat reachability note.
Firewall blocks clients	Confirm the firewall feature is enabled intentionally, the client source IP is correct (including NAT), and send an open command to the trusted control port.
Logs grow quickly	Disable traffic logger/recorder, reduce debug logging, rotate <code>logs/</code> , and review data-retention obligations.
Service continuously restarts	Inspect the application log first. Common causes are unavailable mirror/origin, interface changes with <code>dhcp_check</code> , zero TCP channels beyond the configured wait, and conflicting ports.